

Experiment 1

Aim

To write a Python program to solve the 8-Puzzle Problem using the A* Search Algorithm.

Algorithm

1. Define the initial state and goal state of the 8-puzzle.
2. Use the Manhattan Distance as the heuristic function.
3. Store states in a priority queue based on $f(n) = g(n) + h(n)$.
4. Generate all possible moves of the blank tile (up, down, left, right).
5. Expand the state with the lowest cost.
6. Repeat until the goal state is reached.
7. Display the sequence of moves.

Program

```
from queue import PriorityQueue
```

```
goal = [[1,2,3],[4,5,6],[7,8,0]]
```

```
def h(s):
```

```
    d = 0
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if s[i][j] != 0:
```

```
                x = (s[i][j]-1)//3
```

```
                y = (s[i][j]-1)%3
```

```
                d += abs(x-i) + abs(y-j)
```

```
    return d
```

```

def pos(s):
    for i in range(3):
        for j in range(3):
            if s[i][j] == 0:
                return i, j

def astar(start):
    pq = PriorityQueue()
    pq.put((h(start), 0, start))
    visited = []

    while not pq.empty():
        f, g, s = pq.get()

        if s == goal:
            print("Goal Reached")
            for row in s:
                print(row)
            return

        visited.append(s)

        x, y = pos(s)
        moves = [(1,0),(-1,0),(0,1),(0,-1)]

        for dx, dy in moves:
            nx, ny = x+dx, y+dy
            if 0 <= nx < 3 and 0 <= ny < 3:

```

```
ns = [r[:] for r in s]
ns[x][y], ns[nx][ny] = ns[nx][ny], ns[x][y]
if ns not in visited:
    pq.put((g+1+h(ns), g+1, ns))
```

```
start = [
    [1,2,3],
    [4,0,6],
    [7,5,8]
]
```

astar(start)

Input

Initial State:

1 2 3

4 0 6

7 5 8

Goal State:

1 2 3

4 5 6

7 8 0

Output

```
Goal Reached
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Result

The Python program successfully solved the 8-Puzzle Problem using the A* Search Algorithm and reached the goal state.